

dsh 图像管线深度解析

图片是如何进入大模型的 —— 存储投影传输全链路

面向 AI 应用开发者

deepseek-harness v0.1.1-rc.2

四方对比: dsh · opencode · Codex · Claude Code

目录

1. 基础: 模型到底怎么"看"图
2. 为什么裸发 base64 会出事
3. dsh 全景架构
4. 逐层拆解 (规范化 / 投影缓存 / Files API / 预算闸门 / 坐标换算 / 历史治理)
5. 四方方案对比: dsh vs opencode vs Codex vs Claude Code
6. 实践建议
7. 术语速查

一、基础: 模型到底怎么"看"图

先建立最重要的认知: **大模型本身不认识 PNG/JPEG 文件格式, 它只认识数字矩阵。**

当你说"把这张图发给 GPT", 实际发生的流程是:

你的 PNG 文件 (二进制字节)

| 解码



像素矩阵 [R,G,B] [R,G,B] ... → 模型内部的视觉编码器 (ViT 等)

| 把像素块切成 patch、转成 token



模型"看到"的是一串特殊的图像 token

而**传输协议**层面, 主流只有三张"面孔", 本质都是同一张图的不同包装:

形态	样子	用在哪家
base64 data URL	<code>data:image/png;base64,iVBORw0KGgo...</code>	OpenAI Chat Completions、Claude Messages、DeepSeek、Gemini —— 几乎所有家
file_id 引用	<code>{ "type": "file", "file_id": "file-abc123" }</code>	DeepSeek Files API、OpenAI Files
公网 URL	<code>{ "type": "image_url", "image_url": { "url": "https://..." } }</code>	OpenAI（服务端代抓取）、Gemini、Claude（URL source）

所以回答一个常见疑问：**是的，模型侧最终收到的仍然是 base64 编码的像素**（或等价的 file_id 由服务商自己解码）。所谓“图像管线”，全部工作都发生在**你的客户端把文件字节变成请求体里那串字符之前**——这一段做得好不好，直接决定了成本、速度和可靠性。

1.1 图片是怎么被模型“计费”的

以视觉 token 的通用计算方式为例：

- **OpenAI 系**：图片按 patch 计费。`high` detail 下，一张图先缩放到长边 $\leq 2048\text{px}$ ，再按 512px 正方形切 patch，每个 patch 约 170 token，外加 85 token 基础费；`low` detail 固定 85 token。
- **Anthropic 系**： $\text{token} \approx (\text{宽} \times \text{高}) \div 750$ 。
- **DeepSeek 系**：按归一化后的像素预算折算。

关键结论：图片尺寸直接等于钱和延迟。一张 4K 截图不压缩就发，可能一次就是数千 token；而人眼识别内容往往只需要 1024px。这就是所有智能体都要做降采样的根本原因——**不是怕请求太大，是怕 token 太贵。**

二、为什么裸发 base64 会出事

假设你写了个最朴素的循环对话脚本：每次都把历史消息里的所有图片原样重发。用不了多久你会撞上四堵墙：

墙 1：体积膨胀

base64 编码把 3 字节变 4 字符，净膨胀 33%。一张手机照片 4MB，编码后 5.3MB 文本；一次贴 10 张图，请求体 50MB+。很多网关/代理默认拒绝超过 1~10MB 的请求体。

墙 2: KV-cache 反复失效

这是最隐蔽也最贵的一堵墙。主流 provider 都做 **prefix caching**: 如果本次请求的前 N 个 token 和上次完全一样, 这部分不用重新计算, 费用打一折、延迟大降。但“完全一样”是**字节级**判定。如果你每次对原图重新压缩编码, 压缩器的输出哪怕差一个字节, 从那张图开始往后的所有缓存全部作废。多轮对话里反复引用同一张截图时, 朴素做法等于每轮都在全价重算。

墙 3: 无法重放

会话日志如果把 base64 全存下来, 一天的开发会话日志能到 GB 级。想回看“上周那张架构图”? 翻不动。想在 fork 出的新分支里继续引用旧图? 字节已经不存在了。

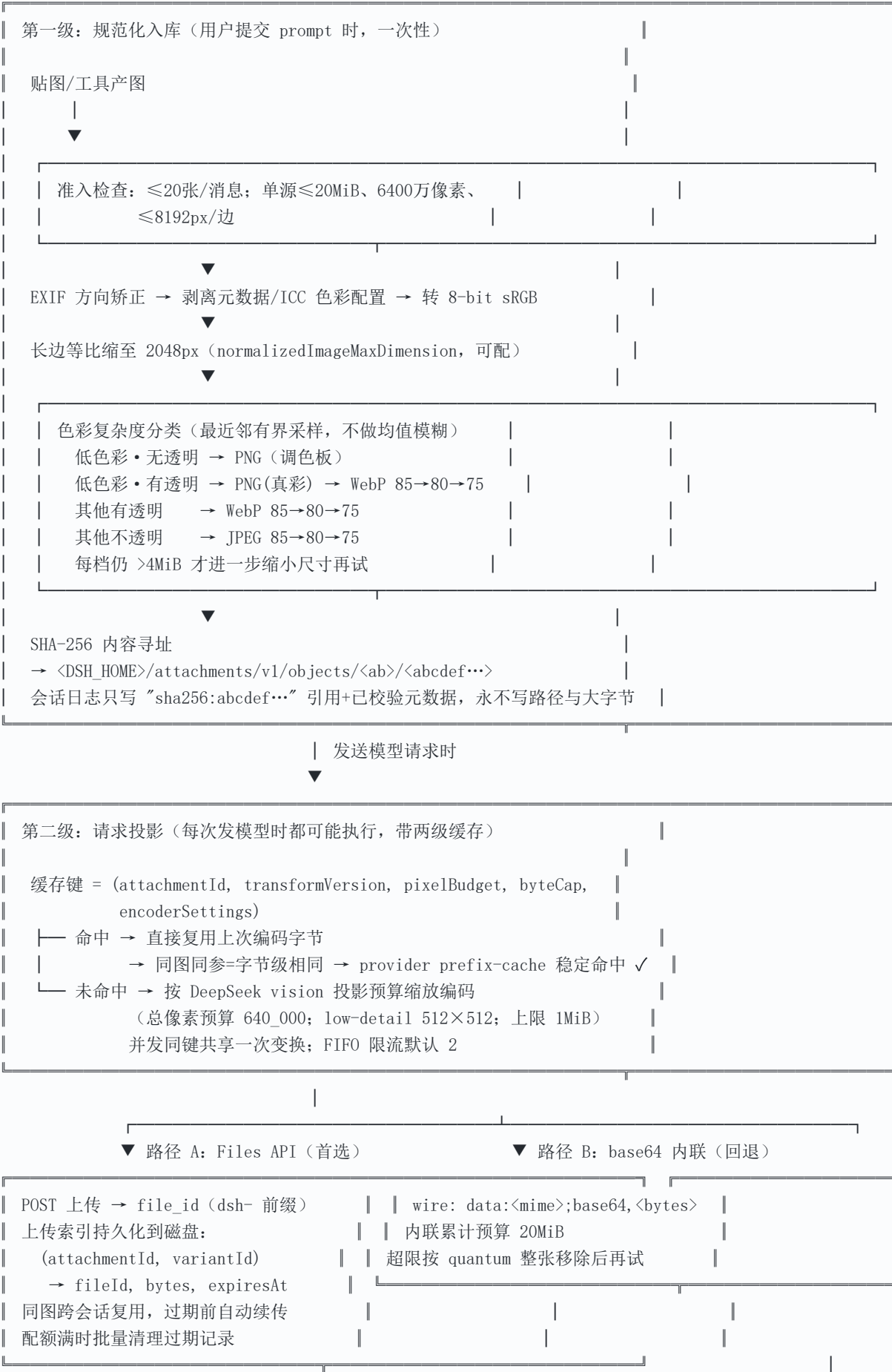
墙 4: 无预算控制

用户一次拖进来 30 张 8K 屏幕截图, 或者某个工具每轮返回一张高清渲染图。没有闸门的话, 要么请求被 provider 拒绝 (Claude 对 >20 张图的请求有更严格的单图尺寸限制), 要么账单爆炸。

dsh 的图像管线就是为了系统性地拆掉这四堵墙。核心思路一句话: 图片存一次 (内容寻址), 发送时按需投影 (带缓存), 传输走引用优先 (file_id), 全链路有预算闸门。

三、dsh 全景架构

dsh 在 **0.1.1-rc.2** 中把图片处理重构为一条三级流水线。先看全景, 再逐级拆解:



```
第三级: wire 格式 (DeepSeek 协议不变) + 预算闸门  
  
{ type:"file", file_id } 或  
{ type:"image_url", image_url:{ url:"data:...;base64,..." } }  
  
闸门: Files 引用累计 ≤128MiB / 内联累计 ≤20MiB /  
      单图编码 ≤1MiB / 单请求 ≤600 张  
超限按 quantum 步长整张剔除 (确定性: 同样输入永远剔除同一批)
```

源码锚点: 第一级 `packages/attachment/attachment-local/src/index.ts` ; 第二级 `request-images.ts` ; 第三级 `llm-deepseek/src/{serialize,file-store,upload-index}.ts` 。

四、逐层拆解

4.1 规范化入库：一张图只认真处理一次

这一级的核心理念是**内容寻址 (Content-Addressed Storage)**：数据的哈希值就是它的地址。改一个像素，哈希全变——天然去重、天然防篡改。

准入即拒绝

每条消息最多 20 张图、累计 200MiB 源字节；单张最大 20MiB、6400 万像素、单边 8192px。超限直接报错，不会默默吞掉。

清洗三连

1. **EXIF 方向矫正**——手机拍照常带旋转标记，不矫正的话模型看到的是横躺的图；
2. **剥除元数据与 ICC 色彩配置**——GPS 坐标、拍摄参数对模型无用还占体积，且避免隐私泄露；
3. **统一 8-bit sRGB/sRGBA**——16-bit 深度、CMYK 等异构色彩空间强制转换，保证后续每一步的字节可重现。

色彩复杂度感知压缩

压缩前先用**最近邻有界采样**判断图片是"低色彩" (截图、图表、二维码) 还是"高色彩" (照片)：

- 低色彩图走 PNG 路线——终端截图常常能压到几十 KB；
- 有透明通道必须保 alpha (WebP/PNG)，绝不能走 JPEG (会变黑底)；
- 照片走 JPEG 逐档降质 (85→80→75)。

每一档质量不行才降下一档，**尺寸缩减只在所有质量档都失败后才进行**——优先保清晰度，其次保体积。

直通优化 (passthrough)

如果原图已经是干净的 8-bit sRGB PNG/JPEG/WebP 且在限制内，**逐字节原样通过**，完全不重编码——零损失零开销的快路径。GIF 动图只取首帧（v1 契约明确排除动画，诚实声明而非静默劣化）。

入库之后

最终字节算 SHA-256 存为对象。会话日志里这张图只剩一行引用 `sha256:e3b0c44298fc...` 加已验证元数据（宽高、媒体类型）。**日志从此与图片体积解耦**。

4.2 请求投影：保护 KV-cache 的核心设计

这是整个管线最有价值的一级。

问题回顾：provider 的 prefix caching 要求请求前缀字节级相同才命中。传统做法每次对原图重新跑一遍压缩，压缩器版本升级、参数微调、甚至并发时序差异都会导致输出字节漂移——前缀从那张图开始全部作废，整段重算。

dsh 的解法：把“图片编码”改造成确定性纯函数，并给结果加缓存。

```
缓存键 = (attachmentId, transformVersion, pixelBudget, byteCap, encoderSettings)
```

五个字段缺一不可：

- `attachmentId`：哪张图（sha256）
- `transformVersion`：变换算法版本——管线升级时递增，旧缓存自然淘汰而不是错误命中
- `pixelBudget / byteCap`：本次请求的预算（DeepSeek vision 默认总像素 640_000，编码上限 1MiB）
- `encoderSettings`：编码器固定参数

同键必同字节，于是：

1. 多轮对话反复引用同一张截图 → 除首轮外，每轮该图的字节**完全一致**
2. provider 的 prefix cache 从头到尾稳定命中 → 长会话成本大幅下降
3. 缩放只在“不放大”前提下进行（小图保持原样），避免无意义放大浪费 token

工程细节：并发展开时多个协程同时要同一张图的投影，共享同一次变换（一个算，其他等）；全局 FIFO 限流器控制同时进行的压缩数量（`imageCompressionConcurrency`，1~8，默认 2）。取消其中一个等待者不会取消共享的工作本身。缓存的字节在复用前还会被完整解码校验一次（确保确实是合法 8-bit sRGB），防止缓存被污染。

4.3 Files API 与 file_id：请求体从 30MB 到几 KB

DeepSeek 提供文件上传服务：先把图 POST 到 Files 端点拿一个 `file_id`，之后聊天请求体里只放这个短字符串，服务端自己去关联存储取图。

rc.2 把这条路径做成**首选**，并补齐了让它生产可用的全部配套：

① 持久化上传索引 (upload-index.ts)

磁盘上的映射表（带 `formatVersion: 3`、原子写 + 文件锁防多进程竞争）：

```
interface DeepSeekUploadRecord {
  scope: DeepSeekFileScope      // 按 apiKey 派生的非秘密命名空间（密钥本身不落盘）
  attachmentId: AttachmentId    // 哪张规范化图
  variantId: ImageVariantIdType // 哪种投影变体（含路由预算）
  fileId: DeepSeekFileIdType    // 远端 file-xxx
  bytes: number
  createdAt / expiresAt        // 过期时间
}
```

效果：同一张图跨会话、跨重启都不再重复上传。第二次用到时查索引直接复用还没过期的 `file_id`；临近过期则按 `refreshMarginSeconds` 自动重传续期。索引的命名空间从 `apiKey` 哈希派生——不同 key 的上传互不污染，且密钥本身永不落盘或进日志。

② 并发去重与取消传播

多个请求同时要传同一张图时，合并为一次上传操作（SharedUpload 模式），大家等同一个 Promise；若所有等待者都取消了且上传尚未完成，才真正 abort 底层请求——避免“取消了一个等待者就把共享上传搞挂”的误伤。

③ 失败回退

Files API 超时（`filesApiTimeoutMs`，rc.2 专门把它与聊天流式超时解耦）或配额错误时，自动降级为路径 B base64 内联。功能不断，只是那次请求变大。

④ 配额回收

上传空间有配额，满了按批次清理已过期的记录再试。

澄清常见误解：file_id 不是给模型“自己去索引图片”用的。模型看到的只是协议字段，取图发生在 DeepSeek 服务端。它的收益纯粹在网络与成本层——请求体从几十 MB 缩到几 KB，且上传一次到处引用。

4.4 预算闸门：确定性剔除，而非分批

层	默认值	保护对象
Files 引用累计	128 MiB	单请求中所有 file_id 代表的原始字节
base64 内联回退累计	20 MiB	回退后的请求体大小
单图投影编码	1 MiB	每张投到请求里的图
单请求图片数	600 张	极端防滥用

超限时按 `quantum` 步长**整张移除图片**——不是把图切片分批发，而是这张图这个请求不带（文本部分照发，图的位置留占位说明）。

为什么强调确定性：剔除逻辑必须满足“同样输入永远剔同样几张”。这样今天被剔的图明天重放会话时还是被剔，日志、实际请求、计费三者永远对得上。随机或依赖时序的剔除策略会让重放失真。

另有一层**上下文溢出联动**：当 provider 返回“上下文窗口超限”错误时，管线绕过常规压力检查，先尝试剔除大图再重试——因为 provider 已经确认了“确实塞不下”，此时无需容量元数据即可行动。

4.5 read_image 坐标换算：视觉工具的地基

rc.2 给 `tool-fs read_image` 工具加了两个返回值：**缩略图的实际尺寸**和**相对原图的坐标比例尺**。

原因：发给模型的永远是按预算缩过的缩略图（比如 640k 像素），但模型经常需要“点击原图某个位置”或“裁剪原图某块区域”。有了比例尺，模型在缩略图上说的“(120, 80)”就能精确换算回原图坐标。这让 GUI 自动化、UI 标注、区域截取类插件（比如 hub 里收录的 modlens）有了标准化地基，不必各自实现一套换算。

配套地，attachment-local 实现了**确定性规范编码**：同一张图的规范化结果永远一致（版本化的转换算法 + 锁定的编码器行为）。这是一切缓存正确性的根基。

4.6 历史图片的上下文管理：老图怎么办

会话越滚越长，早期轮次里的图片占着大量 token。dsh 的做法：

- 投影缓存让“重发同一张图”成本接近零（字节相同，prefix cache 命中）
- 整体逼近上下文上限时，compaction 先于文本处理图片：可选的 `toolResultPruner` 先改写过大的工具结果，摘要再替换掉最老的历史区间
- 单张不可分割的图如果比保留预算还大，明确拒绝压缩并如实上报，而非静默丢图

五、四方方案对比：dsh vs opencode vs Codex vs Claude Code

我们逐一研读了四家的实现：dsh 本仓源码；opencode `packages/opencode/src/image/image.ts`（photon WASM 方案）；OpenAI Codex `codex-rs/core/src/image_preparation.rs` + `codex-rs/utils/image/src/lib.rs` + `compact_remote_v2_images.rs`（Rust 实现）；Claude Code 官方 Vision 文档与其 Read 工具行为。四家面对的协议、用户形态、产品定位各不相同，因此各自演化出了不同但**各自自治**的图像方案。这一章先把每家的方案当作独立设计来讲清楚，再做维度对照。

5.0 四种方案的完整画像

opencode：单点归一化器（photon WASM）

opencode 的图像处理只有一个文件（`image.ts`），职责边界非常清晰：**在发送前把任意图变成 provider 能接受的一份 base64**。

- **解码/缩放引擎用 photon（Rust 编译的 WASM）**——为了跨平台一致性：CLI 要在 macOS/Linux/Windows 上产出完全相同的缩放结果，且以单二进制分发时不需要带原生 sharp/libvips 依赖。典型的“分发形态决定技术选型”。
- **迭代降质策略**：先按上限缩放一次（Lanczos3），然后 PNG 一档 + JPEG 五档质量（80/85/70/55/40）逐个试，哪个先塞进 `maxBase64Bytes`（默认 5MiB）就用哪个；还塞不进就尺寸 $\times 0.75$ 再来一轮，最多 32 轮。
- **可配置**：`attachment.image.{auto_resize, max_width, max_height, max_base64_bytes}` 全部走配置，关掉 `auto_resize` 就变成“超限即报错”。
- **不做事同样明确**：不持久化（data URL 直接进会话消息）、不做 EXIF 矫正、不做跨请求缓存——它面向个人 CLI 单轮交互场景，会话生命周期短，持久化的收益覆盖不了复杂度。

这套设计的自治性在于：**所有状态都交给会话文件和 provider，客户端只做无状态的最后一道归一化**。

Codex：模型上下文管理器（Rust core）

Codex 的图像代码分散在多处，但主线清晰：**图片是上下文预算的一部分，要像管理文本 token 一样管理图片 token**。

- **两套尺寸预算对应两种计费模式**：`HIGH_DETAIL_LIMITS`（长边 2048 / patch ≤ 2500 ，对齐 OpenAI high-detail 计费粒度）与 `UNIFIED_IMAGE_LIMITS`（6000 / 10000），按模型能力与 feature flag 切换——预算直接绑定计费方式。
- **进程内 LRU 编码缓存**（32 条 / 64MiB，sha1 键）：同图同进程重复出现跳过重编码。选择“进程内”而非“磁盘持久”，与其 rollout 文件已保存原始数据的重放策略配套——原始字节不丢，缓存只是加速。
- **`strip_image_details`**：路由到 `responses-lite` 模型时剥掉历史图片的 `detail` 字段——lite 模型按低精度计费，历史图保留高 `detail` 字段纯属浪费。
- **`ImageResizeNotice`**：缩图后生成 `images.resize_notice` 上下文片段，明确告诉模型“第 N/M 张图已从 $W \times H$ 缩至 $W' \times H'$ ”。解决缩略图的真实问题：模型若不知道看的是缩略图，报告的坐标就是错的。
- **原子化 token 截断**：按 token 预算裁剪含图历史时，把“图片+相邻 harness 标签文本”视为不可分割单元，整体保留或整体丢弃，不会撕裂。

这套设计的自治性在于：**OpenAI 协议没有 `file_id` 引用形态可用，图片必然每次随请求重发，所以治理重心放在“每次重发的成本控制”与“模型对自身输入的元认知”上**。

Claude Code：协议原生直发

Claude Code 没有专门的图像预处理管线——依赖 Anthropic Messages API 自身约束体系工作：

- 图片以 base64 source（或 URL source）直接放进 message content，客户端不做转码；
- 遵循 API 侧限制：单图最大 8000×8000px、base64 后 ≤5MB；**单请求 >20 个图片块时触发更严格的每图尺寸限制**（需缩到 2000px 以内否则整请求被拒）；
- 官方文档给出公式化最佳实践：“目标长边 = 1568px 时 token 效率最优”，让调用方自行压缩；
- 历史老图靠开发者自行删除旧 content block 或使用 prompt caching 标记管理。

这套设计的自洽性在于：Anthropic 的 prompt caching 是**显式标记式**的（`cache_control` 由调用方放置），图片块天然可放在缓存断点之后，配合 API 限制报错，把治理责任留在调用方反而灵活。其 Read 工具原生支持读图返回给模型，工具截图走 tool_result 通道与用户贴图共用同一 base64 通路。

dsh：内容寻址的三级流水线

dsh 面向的场景有一个关键差异：**DeepSeek 提供了 Files API（file_id 引用形态），且 dsh 定位为长期运行、会话可 fork 重放的部署型产品。**这两个条件决定了方案形态：

- 服务端支持引用 → 把“上传一次、到处引用”做成一等公民：规范化入库 → 投影 → 上传索引三级流水线（详见第三章）；
- 会话要可审计重放 → 日志必须与图片字节解耦——内容寻址存储让日志只记 `sha256:` 引用；
- 长会话高频多图 → 投影缓存与 KV-cache 字节级稳定性成为核心指标，transformVersion 版本化由此而来。

5.1 协议约束对照

四家设计差异首先来自各自 provider 协议提供了什么：

	传输形态	服务端缓存	可用引用机制
DeepSeek（dsh 目标）	base64 data URL 或 Files API <code>file_id</code>	prefix caching（自动）	有（Files API）
OpenAI（Codex 目标）	Responses API：base64 data URL（input_image）；不支持远程 URL 直发	automatic prefix caching	无通用图片引用
Anthropic（Claude Code 目标）	Messages API：base64 source / URL source	prompt caching（显式标记）	无通用图片引用

三家 provider 都不提供“客户端长期对象存储”原语。**dsh 选择用 DeepSeek Files API 自己补上引用层；Codex/Claude Code 接受“图片随请求重发”的前提，把精力投向重发成本的控制——两条都成立的路线。**

5.2 维度对照（机制描述）

触发时机与处理范围

	处理触发点	处理范围	持久化
dsh	提交时入库 + 每次请求投影	全量规范化 + 投影按预算	双层：对象库 + 上传索引落盘
opcode	仅发送前 normalize 一次	单次缩放/编码	无（结果进会话消息）
Codex	每次组装请求时 prepare	缩放至预算 + detail 决策	进程内 LRU（rollout 存原始数据）
Claude Code	不转换，原样发送	无	无

尺寸与格式策略

	最大边	缩放算法	格式决策	EXIF/色彩
dsh	2048px 入库（可配）；投影按像素预算 640k	sharp/libvips	色彩复杂度三分支逐档	矫正+剥离 +sRGB 强制
opcode	2000×2000	photon WASM Lanczos3	先 PNG 后 JPEG 五档质量	依赖 photon 默认行为
Codex	high 2048 / unified 6000 (patch 双限 2500/10000)	image crate Triangle	统一编码器输出	image crate 解码处理
Claude Code	建议 ≤1568px；>20 图限 2000px	调用方自理	不转换	调用方自理

编码结果缓存

	缓存位置	键设计	生命周期
dsh	本地磁盘投影缓存 + 远端上传索引	(attachmentId, transformVersion, pixelBudget, byteCap, encoderSettings)	持久；transformVersion 升级自然淘汰
opencode	无	—	—
Codex	进程内 LRU 32 条 / 64MiB	sha1(path + mode + limits)	进程退出即失
Claude Code	无	—	—

历史图片的上下文治理

	老图策略	超限行为	对模型的反馈
dsh	compaction 时 pruner 改写大结果后摘要替换老区间；不可分割大图明确拒绝并上报	quantum 步长整张剔除（确定性算法）	剔除记录于会话日志
opencode	无专门机制（历史全量重发）	迭代缩放 32 轮后抛 SizeError	错误含原始/最大尺寸
Codex	lite 路由剥 detail；token 预算截断保证"图+相邻标签"原子去留	占位符替换（按错误类别区分文案）	ImageResizeNotice 片段（第 N/M 张、原尺寸→新尺寸）
Claude Code	开发者手动删旧 block 或用 prompt caching	API 返回 invalid_request_error	无（API 报错信息）

传输路径

	形态	引用复用	回退链路
dsh	Files API file_id 优先, base64 内联回退	持久索引跨会话复用, 过期自动续传	Files 超时/配额错误自动降级内联
opencode	base64 data URL	—	—
Codex	base64 data URL (Responses input_image)	—	—
Claude Code	base64 source (亦支持 URL source)	—	—

5.3 设计取舍解读

四套方案没有高下之分，只有约束不同。核心取舍摆在四条决策轴上：

轴一：状态放哪里？ opencode 和 Claude Code 把状态交给外部（会话文件/provider），客户端无状态——实现简单、无迁移负担。Codex 放进程内存（LRU）。dsh 放磁盘（对象库+上传索引），换来跨会话复用与可审计重放，代价是要处理版本迁移、文件锁、配额回收。

轴二：信任谁做压缩？ Claude Code 完全信任调用方/API（不转换）。opencode 和 dsh 在客户端压缩，但确定性要求不同——dsh 因缓存键含编码输出，必须锁定编码器到字节级；opencode 输出直接消费允许漂移。Codex 用 Rust image crate 取中。

轴三：告诉模型多少？ 大多数方案默认模型不需要知道图片处理历史。Codex 唯一显式反向通知（ImageResizeNotice），源于其场景：编码任务的截图坐标精度直接影响工具调用成败。dsh 的 read_image 坐标比例尺走了同一条思路。

轴四：引用还是内联？ 由 provider 决定。有 Files API（DeepSeek）就有机会做引用复用；没有就只能内联，于是治理重心滑向“如何让每次重发更便宜”——这正是 Codex strip_image_details 与 Claude Code prompt caching 各自的出发点。

理解这四条轴后，再看任何新的智能体产品图像方案，都可以快速定位其在每条轴上的站位，以及站位背后的协议与产品约束。

六、实践建议

给插件开发者

- 不要自己压图。用 `ctx.attachments.saveImages()` 入库拿 `sha256:` 引用，编码/缓存/预算全部由管线承担。
- 做视觉工具时务必利用 `read_image` 返回的坐标比例尺做精确换算，不要假设模型看到的尺寸 = 原图尺寸。
- 工具返回大图前先想想能不能先缩——虽然管线兜底，省下来的就是用户的钱。

给应用架构师

1. 如果产品有多模态历史需求，dsh 的三层模式（内容寻址存储 → 确定性投影缓存 → 引用优先传输）值得整体借鉴，是目前开源智能体里最完整的实现。
2. 无论哪家 API，**保持投影参数恒定**是 prefix-cache 命中率的生命线——预算参数一旦变动，所有历史图片的缓存键全部变化，前缀全灭。

给成本敏感场景

1. 确认 provider 开启了 prefix caching（DeepSeek/OpenAI 自动；Anthropic 需显式 `cache_control`）。
2. 控制单图信息密度：截图尽量裁掉无关边缘；低色彩图（代码截图、终端输出）天然适合 PNG 路线，体积常只有照片的十分之一。
3. 长会话定期主动 compaction，别等被动溢出——被动恢复的代价是整段重算。

七、术语速查

术语	含义
base64 内联	图片二进制经 base64 编码为文本嵌入 JSON body。通用但膨胀 33%
file_id / Files API	图片先上传到服务商文件服务，请求携带短引用 ID，服务端自行取图
KV-cache / prefix caching	服务端缓存请求前缀的计算结果；前缀 字节级相同 才命中，否则从头重算
内容寻址存储	以数据内容的哈希值（SHA-256）作为存储地址，天然去重、写入不可变
确定性投影	相同输入必产生相同输出的编码过程；缓存正确性与重放一致性的前提
transformVersion	变换算法版本号；升级时递增使旧缓存自然淘汰而非错误命中
quantum 剔除	超预算时按固定步长整张移除图片（非切片），保证重放一致性
patch / 视觉 token	模型把图片切分成的小方块，每个 patch 折算固定数量的计费 token
detail 分级	OpenAI 特有：high（高分辨率多 patch） / low（固定 512×512 低耗） / auto
EXIF 方向	手机照片元数据里的旋转标记；不矫正会导致模型看到横躺的图
passthrough	已符合规范的图逐字节原样通过，跳过重编码的快路径
LruCache	最近最少使用缓存；Codex 用它做进程内图片编码缓存（32 条/64MiB）
rollout 文件	Codex 的会话记录格式，包含原始消息项（含图片数据）

基于 deepseek-harness v0.1.1-rc.2 (b150a55)、opencode (photon WASM 实现)、openai/codex (Rust core)、Anthropic Vision 官方文档源码/资料撰写 · 2026-08-24 · dsh-hub 项目
系列下一篇预告：《agent loop 的内部构造》